

Q3 - Sieve of Eratosthenes

Dean Yockey

My computer

My personal computer is a Lenovo ThinkCentre I bought for \$75 on eBay. It's running the Linux Mint operating system. When running `lscpu`, the first few lines read:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:    0-3
```

My computer has 4 CPUs.

Fixing a bug in the original Sieve.c

The original Sieve.c had a bug where when ran with only 1 process, it would report 0 primes. The fix to this was pretty simple.

The original just reads:

```
if (p > 1)
    MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM,
                0, MPI_COMM_WORLD);

/* Stop the timer */
...
```

This means when running with `p == 1`, `global_count` is not set.

The fixed version reads:

```
if (p > 1)
    MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM,
                0, MPI_COMM_WORLD);
```

```

else
    global_count = count;

/* Stop the timer */

```

This results in expected results at 1 processor.

PipedSieve.c

As the assignment just called for "replacing the broadcast step with a pipeline of sends and receives," the changes compared to Sieve.c are not drastic.

At the beginning of the main loop, all processes aside from the first (that is, when `id > 0`) call `MPI_Irecv` to prepare to receive the next prime. The receive it into a new variable, `nextPrime`, so that they can receive the next prime while still using the old prime.

At the end of the loop, processes with `id > 0` wait until the next prime is received. After this, process before the last (`id < p-1`) send the next prime to the process after them.

Program result

The program found that there 6542 primes from 2 to 65535, inclusive.

Benchmarking

I created two separate bash files for benchmarking. They read:

```

#!/bin/bash
# normal Sieve
echo "--- Normal sieve ---"
echo "Q3" > "q3log1.txt"
for p in {1..30}
do
    echo "P: $p" >> q3log1.txt
    echo "P: $p" # so we can monitor progress
    for i in {1..20}
    do
        mpirun -n $p --oversubscribe sieve 65535 >> q3log1.txt
    done
done

```

```

#!/bin/bash
# Pipelined Sieve
echo "--- Pipelined sieve ---"

```

```

echo "Q3" > "q3log2.txt"
for p in {1..30}
do
    echo "P: $p" >> q3log2.txt
    echo "P: $p" # so we can monitor progress
    for i in {1..20}
    do
        mpirun -n $p --oversubscribe psieve 65535 >> q3log2.txt
    done
done

```

I ran the program 20 times for each process count. The output to the logs looked like:

```

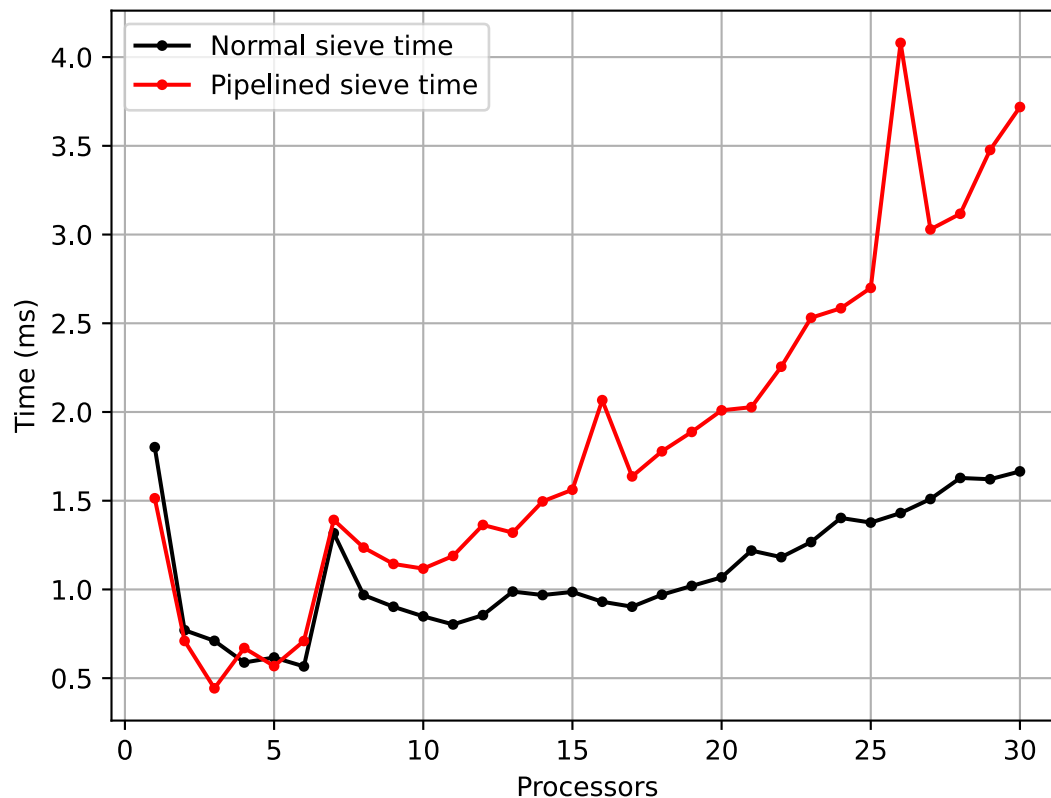
Q3
P: 1
C: 6542 primes under 65535
T: 0.000693
C: 6542 primes under 65535
T: 0.001509
C: 6542 primes under 65535
T: 0.002160
C: 6542 primes under 65535
T: 0.001160
C: 6542 primes under 65535
T: 0.001494
C: 6542 primes under 65535
T: 0.002190
C: 6542 primes under 65535
T: 0.001165
C: 6542 primes under 65535
T: 0.000849
C: 6542 primes under 65535
T: 0.001501
C: 6542 primes under 65535
T: 0.002178
C: 6542 primes under 65535
...

```

I confirmed that all process counts found the same result.

My results

To extract the results from the log file, and visualize them in a line graph, I wrote a python script. I used regex and matplotlib. This is my graph.



Takeaways

The version of the sieve program that uses pipelines overall did much worse than the original version with broadcasts. While the pipeline version does better at 1 processor, this must be just the result of random variation, since they should behave the same at 1 processor.

While the pipeline implementation does mean each process performs less communication, I believe this process also causes later processes to wait longer. In the broadcast version, it's possible for process $n-1$ to receive the next prime before process $n-2$, and therefore get back to work earlier. In the pipeline version, process $n-1$ must always wait for process $n-2$, so it will always get back to work after it.

It's possible a more sophisticated pipeline process would be much more efficient, but as the question just called for pipelining in place of broadcasting, and not a complete overhaul, this will have to do.